27.3 节点 Host 实现

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~270k tokens，输出 ~9k tokens（本节）

前两节分析了节点的概念和 Gateway 侧的管理机制。本节转向节点侧——Node Host 是如何运行的：它如何连接到 Gateway、如何处理命令请求、如何安全地执行系统命令、以及如何作为系统服务持久运行。

27.3.1 Node Host 运行器（`src/node-host/runner.ts`）

启动流程

`runNodeHost` 是 Node Host 的入口函数。它完成配置加载、身份解析、WebSocket 连接建立等一系列初始化工作：

```typescript
// src/node-host/runner.ts（简化）

export async function runNodeHost(opts: NodeHostRunOptions): Promise<void> {
  // 1. 加载或创建配置
  const config = await ensureNodeHostConfig();
  const nodeId = opts.nodeId?.trim() || config.nodeId;
  const displayName = opts.displayName?.trim()
    || config.displayName
    || await getMachineDisplayName();

  // 2. 保存 Gateway 连接信息
  const gateway: NodeHostGatewayConfig = {
    host: opts.gatewayHost,
    port: opts.gatewayPort,
    tls: opts.gatewayTls ?? loadConfig().gateway?.tls?.enabled ?? false,
    tlsFingerprint: opts.gatewayTlsFingerprint,
  };
  config.gateway = gateway;
  await saveNodeHostConfig(config);

  // 3. 解析浏览器Agent 配置
  const browserProxy = resolveBrowserProxyConfig();
  const browserProxyEnabled = browserProxy.enabled && resolvedBrowser.enabled;

  // 4. 解析认证信息
  const token = process.env.OPENCLAW_GATEWAY_TOKEN?.trim()
    || cfg.gateway?.auth?.token;
  const password = process.env.OPENCLAW_GATEWAY_PASSWORD?.trim()
    || cfg.gateway?.auth?.password;

  // 5. 确保 PATH 环境变量可用
  const pathEnv = ensureNodePathEnv();

  // 6. 创建 Gateway 客户端并连接
  const client = new GatewayClient({
    url: `${gateway.tls ? "wss" : "ws"}://${host}:${port}`,
    token, password,
    instanceId: nodeId,
    clientName: GATEWAY_CLIENT_NAMES.NODE_HOST,
    clientDisplayName: displayName,
    mode: GATEWAY_CLIENT_MODES.NODE,
    role: "node",
    caps: ["system", ...(browserProxyEnabled ? ["browser"] : [])],
```

```
    commands: [
      "system.run", "system.which",
      "system.execApprovals.get", "system.execApprovals.set",
      ...(browserProxyEnabled ? ["browser.proxy"] : []),
    ],
    pathEnv,
    deviceIdentity: loadOrCreateDeviceIdentity(),
    onEvent: (evt) => {
      if (evt.event !== "node.invoke.request") return;
      const payload = coerceNodeInvokePayload(evt.payload);
      if (!payload) return;
      void handleInvoke(payload, client, skillBins);
    },
    // ...
  });

  client.start();
  await new Promise(() => {}); // 永不 resolve——保持进程运行
}
```

启动流程的几个关键设计点：

步骤	设计意图
`ensureNodeHostConfig`	如果配置文件不存在，自动生成 nodeId（UUID）并保存
`getMachineDisplayName`	从操作系统获取友好名称（如 "MacBook Pro"）
`ensureNodePathEnv`	确保 OpenClaw CLI 在 PATH 中，以便被其他工具调用
`loadOrCreateDeviceIdentity`	生成持久的设备标识，用于配对和身份验证
`await new Promise(() => {})`	经典的"永不退出"模式——保持事件循环运行

声明能力与命令

注意 `caps` 和 `commands` 的区别：

- **`caps` （能力）**：粗粒度的能力分类，如 `"system"` 表示可以执行系统命令、 `"browser"` 表示可以代理浏览器操作。
- **`commands` （命令）**：细粒度的命令列表，如 `"system.run"` 表示支持执行 Shell 命令。

Node Host 默认声明支持 `system.*` 类命令。如果配置了浏览器代理，还会额外声明 `browser.proxy` 。

事件监听

Node Host 通过 `onEvent` 回调监听 Gateway 推送的事件。收到 `node.invoke.request` 事件时，它将事件 payload 传递给 `handleInvoke` 处理：

```
onEvent: (evt) => {
  if (evt.event !== "node.invoke.request") return;
  const payload = coerceNodeInvokePayload(evt.payload);
  if (!payload) return;
  void handleInvoke(payload, client, skillBins);
},
```

`coerceNodeInvokePayload` 对事件数据做防御性校验——即使收到格式不正确的数据也不会崩溃：

```
function coerceNodeInvokePayload(payload: unknown): NodeInvokeRequestPayload | null {
  if (!payload || typeof payload !== "object") return null;
  const obj = payload as Record<string, unknown>;
  const id = typeof obj.id === "string" ? obj.id.trim() : "";
  const nodeId = typeof obj.nodeId === "string" ? obj.nodeId.trim() : "";
  const command = typeof obj.command === "string" ? obj.command.trim() : "";
  if (!id || !nodeId || !command) return null;
  // 支持 paramsJSON 字符串和 params 对象两种格式
  const paramsJSON = typeof obj.paramsJSON === "string"
    ? obj.paramsJSON
    : obj.params !== undefined ? JSON.stringify(obj.params) : null;
  return { id, nodeId, command, paramsJSON, timeoutMs, idempotencyKey };
}
```

27.3.2 命令处理分发

`handleInvoke` 是 Node Host 的命令路由器，根据 `command` 字段将请求分发到不同的处理逻辑：

```
// src/node-host/runner.ts（简化）

async function handleInvoke(
  frame: NodeInvokeRequestPayload,
  client: GatewayClient,
  skillBins: SkillBinsCache,
) {
  const command = String(frame.command ?? "");

  switch (command) {
    case "system.execApprovals.get":
      // → 返回当前执行审批配置
    case "system.execApprovals.set":
      // → 更新执行审批配置（带乐观锁）
    case "system.which":
      // → 查找二进制文件路径
    case "browser.proxy":
      // → 代理浏览器操作
    case "system.run":
      // → 执行系统命令（最复杂的分支）
    default:
      // → 返回 "command not supported"
  }
}
```

system.which —— 二进制文件发现

`system.which` 命令查找指定二进制文件的绝对路径，类似于 Unix 的 `which` 命令：

```
async function handleSystemWhich(
  params: SystemWhichParams,
  env?: Record<string, string>,
) {
  const found: Record<string, string> = {};
  for (const bin of params.bins) {
    const path = resolveExecutable(bin.trim(), env);
    if (path) found[bin] = path;
  }
  return { bins: found };
}
```

`resolveExecutable` 遍历 PATH 中的每个目录，在 Windows 上还会尝试添加 `.EXE` 、 `.CMD` 等后缀。这个命令被 Agent 用来在执行命令前**探测目标环境**——例如检查 `python3` 、 `node` 、 `git` 是否可用。

browser.proxy —— 浏览器操作代理

当 Node Host 配置了浏览器代理时，Gateway 可以通过 `browser.proxy` 命令远程控制 Node Host 机器上的浏览器。实现使用了第 16 章介绍的浏览器控制服务：

```
// src/node-host/runner.ts（简化）

case "browser.proxy": {
  // 安全检查：浏览器代理是否启用
  if (!proxyConfig.enabled) throw new Error("node browser proxy disabled");

  // 安全检查：请求的 profile 是否在允许列表中
  if (!isProfileAllowed({ allowProfiles, profile: requestedProfile })) {
    throw new Error("browser profile not allowed");
  }

  // 确保浏览器控制服务已启动
  await ensureBrowserControlService();

  // 将请求分发到浏览器路由
  const dispatcher = createBrowserRouteDispatcher(createBrowserControlContext());
  const response = await withTimeout(
    dispatcher.dispatch({ method, path, query, body }),
    params.timeoutMs,
  );

  // 如果响应包含文件路径，读取并 base64 编码
  const paths = collectBrowserProxyPaths(response.body);
  const files = await Promise.all(paths.map(readBrowserProxyFile));

  await sendInvokeResult(client, frame, { ok: true, payloadJSON: JSON.stringify({ resul
}
```

文件（如截图）通过 base64 编码内联传输，单个文件不超过 10MB（`BROWSER_PROXY_MAX_FILE_BYTES`）。

27.3.3 system.run —— 安全的命令执行

`system.run` 是 Node Host 最核心也最危险的能力——它允许 Agent 在远程机器上执行任意 Shell 命令。因此，它的安全控制是整个节点系统中最复杂的部分。

安全层次

命令执行经过四层安全检查：

```
┌──────────────────────────────────────────────────────────────┐
│  ① macOS App 执行主机（Exec Host）                              │
│  → 通过 Unix Socket 委托给 macOS 原生应用执行                    │
│  → 如果可用，优先使用此路径                                      │
├──────────────────────────────────────────────────────────────┤
│  ② 安全等级（Security Level）                                   │
│  → "deny"：禁止一切执行                                         │
│  → "allowlist"：仅允许白名单内的命令                             │
│  → "full"：允许一切执行                                         │
├──────────────────────────────────────────────────────────────┤
│  ③ 审批机制（Approval）                                         │
│  → "off"：无需审批                                              │
│  → "on-miss"：白名单未命中时需要审批                             │
│  → "always"：每次都需要审批                                     │
├──────────────────────────────────────────────────────────────┤
│  ④ 白名单评估（Allowlist Evaluation）                           │
│  → 解析命令 argv → 匹配已授权的可执行文件路径                    │
│  → 支持 Skill Bins（动态白名单）                                 │
└──────────────────────────────────────────────────────────────┘
```

macOS 执行主机优先

在 macOS 上，Node Host 优先尝试通过 **Exec Host**（macOS 原生应用的执行沙箱）来执行命令：

```
// src/node-host/runner.ts（简化）

const useMacAppExec = process.platform === "darwin";
if (useMacAppExec) {
  const response = await runViaMacAppExecHost({ approvals, request: execRequest });
  if (response && response.ok) {
    // macOS App 成功执行
    await sendNodeEvent(client, "exec.finished", { ... });
    await sendInvokeResult(client, frame, { ok: true, payloadJSON: JSON.stringify(resul
    return;
  }
  if (response && !response.ok) {
    // macOS App 拒绝执行
    await sendNodeEvent(client, "exec.denied", { reason: response.error.reason });
    await sendInvokeResult(client, frame, { ok: false, error: { message: response.error
    return;
  }
  // response === null: macOS App 不可达
  if (execHostEnforced || !execHostFallbackAllowed) {
    // 强制模式：不回退，直接拒绝
    await sendInvokeResult(client, frame, { ok: false, error: { code: "UNAVAILABLE" } }
    return;
  }
  // 否则：回退到直接执行
}
```

> **衍生解释：Exec Host 与 Unix Socket**
>
> Exec Host 是 OpenClaw macOS 原生应用提供的命令执行服务。Node Host 通过 **Unix Domain Socket** 与它通信。Unix Socket 是一种本机进程间通信（IPC）机制，比 TCP/IP 快得多（无需网络协议栈），且天然限制在本机，安全性更高。 `OPENCLAW_NODE_EXEC_HOST=app` 环境变量可以强制要求所有执行必须经过 Exec Host，防止命令绕过 macOS 应用的审批 UI。

白名单评估流程

当安全等级为 `allowlist` 时，命令需要通过白名单评估：

```
// src/node-host/runner.ts（简化）

// 两种评估路径：
if (rawCommand) {
  // Shell 命令字符串 → evaluateShellAllowlist
  const result = evaluateShellAllowlist({
    command: rawCommand,
    allowlist: approvals.allowlist,
    safeBins,                    // 内置安全二进制列表
    skillBins: await skillBins.current(), // 技能动态白名单
    autoAllowSkills,
    platform: process.platform,
  });
} else {
  // argv 数组 → analyzeArgvCommand + evaluateExecAllowlist
  const analysis = analyzeArgvCommand({ argv, cwd, env });
  const result = evaluateExecAllowlist({
    analysis,
    allowlist: approvals.allowlist,
    safeBins,
    skillBins,
    autoAllowSkills,
  });
}
```

Skill Bins 是一个有趣的机制——技能系统可以动态声明一些二进制文件为"安全的"，这些二进制文件会自动加入白名单。 `SkillBinsCache` 以 90 秒 TTL 缓存这个列表：

```
class SkillBinsCache {
  private bins = new Set<string>();
  private lastRefresh = 0;
  private readonly ttlMs = 90_000;

  async current(force = false): Promise<Set<string>> {
    if (force || Date.now() - this.lastRefresh > this.ttlMs) {
      await this.refresh();
    }
    return this.bins;
  }

  private async refresh() {
    const bins = await this.fetch(); // 向 Gateway 请求 skills.bins
    this.bins = new Set(bins);
    this.lastRefresh = Date.now();
  }
}
```

命令执行

通过所有安全检查后，命令通过 `runCommand` 函数实际执行：

```
// src/node-host/runner.ts（简化）

async function runCommand(
  argv: string[],
  cwd: string | undefined,
  env: Record<string, string> | undefined,
  timeoutMs: number | undefined,
): Promise<RunResult> {
  return await new Promise((resolve) => {
    let stdout = "", stderr = "";
    let outputLen = 0, truncated = false, timedOut = false;

    const child = spawn(argv[0], argv.slice(1), {
      cwd, env,
      stdio: ["ignore", "pipe", "pipe"],
      windowsHide: true,
    });

    // 输出捕获（带 200KB 截断保护）
    const onChunk = (chunk: Buffer, target: "stdout" | "stderr") => {
      if (outputLen >= OUTPUT_CAP) { truncated = true; return; }
      const remaining = OUTPUT_CAP - outputLen;
      const slice = chunk.length > remaining ? chunk.subarray(0, remaining) : chunk;
      // ...
    };

    // 超时处理
    if (timeoutMs > 0) {
      setTimeout(() => { timedOut = true; child.kill("SIGKILL"); }, timeoutMs);
    }

    child.on("exit", (code) => resolve({ exitCode: code, timedOut, success: code === 0,
  });
}
```

关键的安全措施：

措施	实现	目的
输出截断	`OUTPUT_CAP = 200_000` 字符	防止内存溢出
超时终止	`SIGKILL`	防止命令永远运行

措施	实现	目的
环境变量清洗	`sanitizeEnv()`	阻止注入危险变量
stdin 关闭	`stdio: ["ignore", ...]`	防止交互式命令阻塞
Windows 隐藏	`windowsHide: true`	防止弹出控制台窗口

环境变量安全

`sanitizeEnv` 函数过滤掉危险的环境变量：

```
// src/node-host/runner.ts

const blockedEnvKeys = new Set([
  "NODE_OPTIONS",    // 可注入任意 Node.js 参数
  "PYTHONHOME",      // 可劫持 Python 解释器
  "PYTHONPATH",      // 可注入 Python 模块路径
  "PERL5LIB",       // 可注入 Perl 库路径
  "PERL5OPT",       // 可注入 Perl 选项
  "RUBYOPT",        // 可注入 Ruby 选项
]);

const blockedEnvPrefixes = [
  "DYLD_",  // macOS 动态链接器控制
  "LD_",    // Linux 动态链接器控制
];
```

> **衍生解释：环境变量注入攻击**
>
> 许多编程语言的运行时会读取环境变量来改变行为。例如 `LD_PRELOAD` 可以让 Linux 在加载任何程序时先加载一个自定义的共享库，从而劫持系统调用。 `NODE_OPTIONS` 可以给 Node.js 注入 `--require` 参数来加载恶意脚本。阻止这些变量的覆盖是应用层安全的基本实践。

执行事件上报

每次命令执行（无论成功、失败还是被拒绝），Node Host 都会向 Gateway 发送事件通知：

```
async function sendNodeEvent(client: GatewayClient, event: string, payload: unknown) {
  try {
    await client.request("node.event", {
      event,
      payloadJSON: payload ? JSON.stringify(payload) : null,
    });
  } catch {
    // 事件发送是尽力而为，不阻塞主流程
  }
}
```

事件输出被截断到 20KB（`OUTPUT_EVENT_TAIL`），避免大量输出淹没 WebSocket 通道。

27.3.4 节点配置（`src/node-host/config.ts`）

配置结构

Node Host 的配置存储在状态目录下的 `node.json` 文件中：

```
// src/node-host/config.ts

export type NodeHostConfig = {
  version: 1;                        // 配置版本号（用于迁移）
  nodeId: string;                    // 节点唯一标识
  token?: string;                    // 配对令牌（可选）
  displayName?: string;              // 显示名称
  gateway?: NodeHostGatewayConfig;   // Gateway 连接信息
};

export type NodeHostGatewayConfig = {
  host?: string;           // Gateway 地址，默认 127.0.0.1
  port?: number;           // Gateway 端口，默认 18789
  tls?: boolean;           // 是否使用 TLS
  tlsFingerprint?: string; // TLS 证书指纹（用于自签证书）
};
```

安全写入

配置文件使用 `0o600` 权限写入，确保只有当前用户可读写：

```
export async function saveNodeHostConfig(config: NodeHostConfig): Promise<void> {
  const filePath = resolveNodeHostConfigPath();
  await fs.mkdir(path.dirname(filePath), { recursive: true });
  const payload = JSON.stringify(config, null, 2);
  await fs.writeFile(filePath, `${payload}\n`, { mode: 0o600 });
  try {
    await fs.chmod(filePath, 0o600); // 双重保障
  } catch {
    // Windows 不支持 chmod
  }
}
```

自动初始化

`ensureNodeHostConfig` 实现了"读取-规范化-保存"的原子操作：

```
export async function ensureNodeHostConfig(): Promise<NodeHostConfig> {
  const existing = await loadNodeHostConfig();   // 读取（可能为 null）
  const normalized = normalizeConfig(existing);  // 规范化（生成缺失字段）
  await saveNodeHostConfig(normalized);          // 保存
  return normalized;
}
```

`normalizeConfig` 确保必填字段存在——如果 `nodeId` 为空，自动生成一个 UUID。

27.3.5 CLI 与守护进程

两套 CLI

OpenClaw 提供两套节点相关的 CLI 命令：

CLI 命令组	用途	入口文件
`openclaw node`	Node Host 管理	`src/cli/node-cli/register.ts`

CLI 命令组	用途	入口文件
`openclaw nodes`	节点远程管理	`src/cli/nodes-cli/register.ts`

`openclaw node` 管理本机的 Node Host 进程：

```
openclaw node run                  # 前台运行
openclaw node install              # 安装为系统服务
openclaw node uninstall            # 卸载系统服务
openclaw node status               # 查看服务状态
openclaw node stop                 # 停止服务
openclaw node restart              # 重启服务
```

`openclaw nodes` 通过 Gateway RPC 远程管理节点：

```
openclaw nodes list                # 列出所有节点
openclaw nodes describe <nodeId>   # 查看节点详情
openclaw nodes invoke <nodeId> ... # 向节点发送命令
openclaw nodes pair ...            # 配对管理
openclaw nodes canvas ...          # Canvas 操作
openclaw nodes camera ...          # 摄像头操作
openclaw nodes screen ...          # 录屏操作
openclaw nodes location ...        # 定位操作
openclaw nodes notify ...          # 系统通知
```

守护进程安装

`openclaw node install` 将 Node Host 安装为操作系统的后台服务。安装计划（Install Plan）会根据当前操作系统选择对应的服务管理器：

操作系统	服务管理器	服务文件
macOS	launchd	`~/Library/LaunchAgents/ai.openclaw.node.plist`
Linux	systemd	`~/.config/systemd/user/openclaw-node.service`
Windows	schtasks	`OpenClaw-Node` 计划任务

安装时支持多个选项：

```
openclaw node install \
  --host 192.168.1.100 \        # 远程 Gateway 地址
  --port 18789 \                # Gateway 端口
  --tls \                       # 启用 TLS
  --tls-fingerprint <sha256> \ # 自签证书指纹
  --node-id <id> \              # 指定节点 ID
  --display-name "My Server" \ # 显示名称
  --runtime bun \               # 使用 Bun 运行时（默认 Node.js）
  --force                       # 强制重装
```

本节小结

1. **Node Host 是一个长连接客户端**——通过 `GatewayClient` 建立 WebSocket 连接，声明自身的能力和支持的命令，然后持续监听 `node.invoke.request` 事件。
2. **命令分发遵循 switch-case 路由**—— `handleInvoke` 根据命令名称分发到 `system.run` 、 `system.which` 、 `browser.proxy` 等处理函数。
3. **system.run 有四层安全防护**——macOS Exec Host 优先 → 安全等级检查 → 审批机制 → 白名单评估，确保远程命令执行不会成为安全漏洞。
4. **环境变量被严格清洗**——阻止 `NODE_OPTIONS` 、 `LD_PRELOAD` 、 `DYLD_*` 等危险变量的注入。
5. **输出有双重截断**——执行输出限制 200KB，事件上报输出限制 20KB，防止大量输出冲垮通信链路。
6. **配置文件权限严格**—— `0o600` 确保只有当前用户可读写，token 等敏感信息不会泄露。
7. **跨平台守护进程支持**——macOS 使用 launchd、Linux 使用 systemd、Windows 使用 schtasks，统一的 CLI 接口屏蔽了平台差异。